

Installation

The setup of PyCRAM can be divided in four steps:

- Install ROS
- Installing Dependencies
- Cloning the PyCRAM repo
- Building your ROS workspace

All dependencies are available via PyPi.

PyCRAM is developed and tested currently with Python3.8, Ubuntu 20.04 and ROS Noetic.

This guide expects you to have a GitHub account with an SSH key (you can read about adding a new ssh key [here](#)).

Installing ROS

PyCRAM uses ROS for a variety of functionality, for this reason you need a working ROS installation on your machine. For information on how to install ROS please refer to the official documentation [here](#) for ROS1 Noetic or [here](#) for ROS2 Foxy.

Installing Dependencies

The dependencies you will need are:

- Pip
- vcstool
- curl

These are available via the Ubuntu apt-repos and can be installed via the terminal:

```
sudo apt-get install python3-pip python3-vcstool curl
```

Using the install script

The easiest way to install PyCRAM is to use the install script. This script will install all necessary dependencies and clone the PyCRAM repository into your ROS workspace. Furthermore, it will setup the Python environment for PyCRAM and install the necessary Python packages.

Additionally the script can install the IPython startup scripts which come in handy when using PyCRAM in an IPython shell for development.

To use the script you have to setup a ROS workspace before:

Now you can run the install script, *The scripts excepts that it is executed from /workspace/ros/src:*

```
curl -s
https://raw.githubusercontent.com/cram2/pycram/dev/scripts/install.
sh | bash
```

PyCRAM on Ubuntu24.04 (ROS Jazzy)

Before installing PyCRAM you need to setup a ROS workspace into which PyCRAM can be cloned.

```
mkdir -p ~/workspace/ros/src
cd workspace/ros
colcon build --symlink-install
source install/setup.bash
```

If `colcon build` does not work this probably means that you did not source your ROS installation. Source it by invoking:

```
source /opt/ros/jazzy/setup.bash
```

Now you can install PyCRAM into your ROS workspace.

```
cd ~/workspace/ros/src
vcs import --input
https://raw.githubusercontent.com/cram2/pycram/dev/pycram-
ros2.rosinstall --recursive
rosdep update
rosdep install --ignore-src --from-paths . -r
cd ..
colcon build --symlink-install
source devel/setup.bash
echo "source ~/workspace/ros/install/setup.bash" >> ~/.bashrc
```

Afterwards continue with the steps under [Python Dependencies](#).

PyCRAM on Ubuntu 20.04 (ROS Noetic)

Before installing PyCRAM you need to setup a ROS workspace into which PyCRAM can be cloned.

```
mkdir -p ~/workspace/ros/src
cd workspace/ros
catkin build
source devel/setup.bash
```

If `catkin build` does not work this probably means that you did not source your ROS installation. Source it by invoking:

```
source /opt/ros/noetic/setup.bash
```

Now you can install PyCRAM into your ROS workspace.

```
cd ~/workspace/ros/src
vcs import --input
https://raw.githubusercontent.com/cram2/pycram/dev/pycram.rosinstal
l --recursive
rosdep update
rosdep install --ignore-src --from-paths . -r
cd ..
catkin build
source devel/setup.bash
echo "source ~/workspace/ros/devel/setup.bash" >> ~/.bashrc
```

The cloning and setting up can take several minutes. After the command finishes you should see a number of repositories in your ROS workspace.

The cloned repository contains the source code for PyCRAM as well as two short demos which demonstrate how to use it.

Python Dependencies

To install the Python dependencies Pip is used. To install Pip type the following command into a terminal.

```
sudo apt-get install python3-pip
```

Now the actual Python packages can be installed, these are summarized in the `requirements.txt` in the PyCRAM repo. For this first navigate to your PyCRAM repo.

```
cd ~/workspace/ros/src/pycram
```

Then install the Python packages in the `requirements.txt` file

```
sudo pip3 install -r requirements.txt
```

This installs the packages into `/usr/local/lib`. If you prefer to not clutter your system-wide python installation, you can also install the packages into the catkin workspace as follows:

```
# install packages into catkin workspace instead of ~/.local
export PYTHONUSERBASE=~/workspace/ros/devel
# don't install packages that are available in system
export PIP_IGNORE_INSTALLED=0
```

```
pip3 install -r requirements.txt
pip3 install -r src/neem_interface_python/requirements.txt
```

Building your ROS workspace

Building and sourcing your ROS workspace using catkin compiles all ROS packages and manages the appending to the respective PATH variables. This is necessary to be able to import PyCRAM via the Python import system and to find the robot descriptions in the launch file.

If you have been following the tutorial steps until now you can skip this part.

You can build your ROS workspace with the following commands:

```
cd ~/workspace/ros
catkin build
source devel/setup.bash
```

Using PyCRAM

To start using PyCRAM you first need to launch the ROS launch file. This launchfile is located in the directory “launch” and is named “ik_and_description.launch”.

The launchfile can be started with the following command:

```
roslaunch pycram ik_and_description.launch
```

What the launch file does is start a ROS master, upload the robot URDF to the parameter server as well as starting the IK solver.

Building the documentation

The documentation uses jupyter-book as engine. Building the documentation requires Python 3.9 to avoid dependency conflicts. To install Python 3.9 on Ubuntu 20.04, use the following commands:

```
sudo apt install python3.9
```

It is recommended to use a virtual environment to avoid conflicts with the system Python.

```
apt-get install python3-virtualenv
virtualenv -p python3.9 --system-site-packages build-doc
source build-doc/bin/activate
```

Install the requirements in your python interpreter.

```
cd ~/workspace/ros/src/pycram/doc
pip install -r requirements.txt
```

Run pycram and build the docs.

```
cd ~/workspace/ros
roslaunch pycram ik_and_description.launch
cd src/pycram/doc/source
jupyter-book build .
```

Show the index.

```
firefox _build/html/index.html
```

Setting up PyCRAM with PyCharm

Setting up PyCharm with packages that rely on rospy is non trivial. Follow this guide to get correct syntax highlighting for the PyCRAM project.

Install PyCharm Professional

First, [install PyCharm Professional](#).

Create a JetBrains account and verify it for educational purpose. Normally, a school email address would suffice, otherwise you would have to upload your student/employee id card. The verification process typically takes 1~2-week time, so until then please use Trial version. Once your account is verified, you can unlock the PyCharm Professional features in PyCharm.

The next step will set up the virtual Python environment, so it can be used as a project interpreter in PyCharm.

Set up virtualenv

The virtualenvwrapper allows to manage virtual Python environments, where additional packages can be installed without the risk of breaking the system-wide Python configuration. Install [virtualenvwrapper](#) via pip and set it up.

```
sudo pip3 install virtualenvwrapper
```

(Optional but recommended) Set virtualenvwrapper's *WORKON_HOME* env variable, of which the default value is *~/virtualenvs*

```
echo "export WORKON_HOME=~/.envs" >> ~/.bashrc
mkdir -p $WORKON_HOME
```

Activate virtualenvwrapper at terminal start

```
echo "source /usr/local/bin/virtualenvwrapper.sh" >> ~/.bashrc
source ~/.bashrc
```

Create a virtual env based on the workspaces libraries (see [build-ws](#)) and add the `–system-site-packages` to get them properly. The env will be registered in `$WORKON_HOME`.

```
source ~/workspace/ros/devel/setup.bash
mkvirtualenv pycram --system-site-packages
ls $WORKON_HOME
```

Check if the ROS libraries are available in the virtual env.

```
workon pycram
python -c "import rospy"
```

If it complains about *python*, install the following, to set *python* to Python 3 by default.

```
sudo apt install python-is-python3
```

If it finds *python* but complains about missing packages, make sure that the workspace is sourced before creating the virtual env. Also remember to create the virtual env with the `–system-site-packages` flag.

If this returns no errors, you can be sure that *rospy* is usable in your virtual environment. Next you have to build the ros workspace including *pycram* and source it as described in [build-ws](#).

Configure PyCharm

Always start PyCharm from the terminal via

```
pycharm-professional
or
~/pycharm/bin/pycharm.sh
```

Select **File | Open** and select the root folder of the PyCRAM package. Next go to **File | Settings | Project: pycram | Python Interpreter** and set up your virtual environment with *rospy* and the sourced workspace available as the python interpreter.

Finally, go to **File | Settings | Project: pycram | Project Structure** and mark the `src` folder as Sources, the test folder as Tests and the resources as Resources.

To verify that it works, you can execute any Testcase.

Useful tips

- [Keyboard shortcuts](#)
- [Keymap](#), which can be configured in **Settings | Keymap**. The default is GNOME.
- [Python interpreter](#)
- [Python virtual environment](#)
- [Python packages](#)
- [Python console](#)
- **View | Active Editor | Soft-wrap**: wrap text inside the editor view
- **View | Tool Windows | Structure**: display structure window for easy content navigation
- **F12**: Open terminal
- **Double Shift**: Quick file search
- **Alt + Shift + 1**: Reveal/Select current file in Project View
- **Ctrl F/R**: Find/Replace text in current file
- **Ctrl Shift F/R**: Find/Replace text in the whole project, module, directory, scope
- **Settings | Editor | Inspections | Code is compatible with specific Python versions**: Enable/Disable Python version-specific warnings

Using IPython as REPL

If you want to use a REPL with PyCRAM you can use IPython for that. IPython can be installed via the Ubuntu package manager.

```
sudo apt install ipython3
```

Enable autoreload

To use changes made in the Python file while the Repl is running you need to enable the iPython extension `autoreload`. This can be done using the iPython startup files, these are files which are always run if iPython is started. First run `ipython profile create` to create a [default profile](#). Then you will find the startup files located in `~/.ipython/profile_default/startup` along with a README file which explains the usage of the startup files. In this directory create a file called `00-autoreload.ipynb` and enter the following code to the file.

```
%load_ext autoreload
%autoreload 2
```

The first line loads the extension to iPython and the second line configures autoreload to reload all modules before the code in the shell is executed.

Run scripts

IPython allows to run Python files and enables the access to created variables. This can be helpful if you want to create a setup script which initializes things like the BulletWorld, Objects and imports relevant modules.

To execute a Python script simply run `run filename.py` in the IPython shell.

Here is an example how a setup script can look like.

```
from pycram.bullet_world import BulletWorld, Object
from pycram.designators.action_designator import *
from pycram.designators.motion_designator import *
from pycram.designators.location_designator import *
from pycram.designators.object_designator import *
from pycram.process_module import simulated_robot
from pycram.pose import Pose
from pycram.enums import ObjectType

world = BulletWorld()

robot = Object("pr2", ObjectType.ROBOT, "pr2.urdf")
kitchen = Object("kitchen", ObjectType.ENVIRONMENT, "kitchen.urdf")
cereal = Object("cereal", ObjectType.BREAKFAST_CEREAL,
"breakfast_cereal.stl", pose=Pose([1.4, 1, 0.9
```